讲义 6_Flask 简明教程

Flask 定义

Flask 是一个用 Python 编写的轻量级 Web 应用框架。

Flask 基于 WSGI（Web Server Gateway Interface）和 Jinja2 模板引擎，旨在帮助开发者快速、简便地创建 Web 应用。

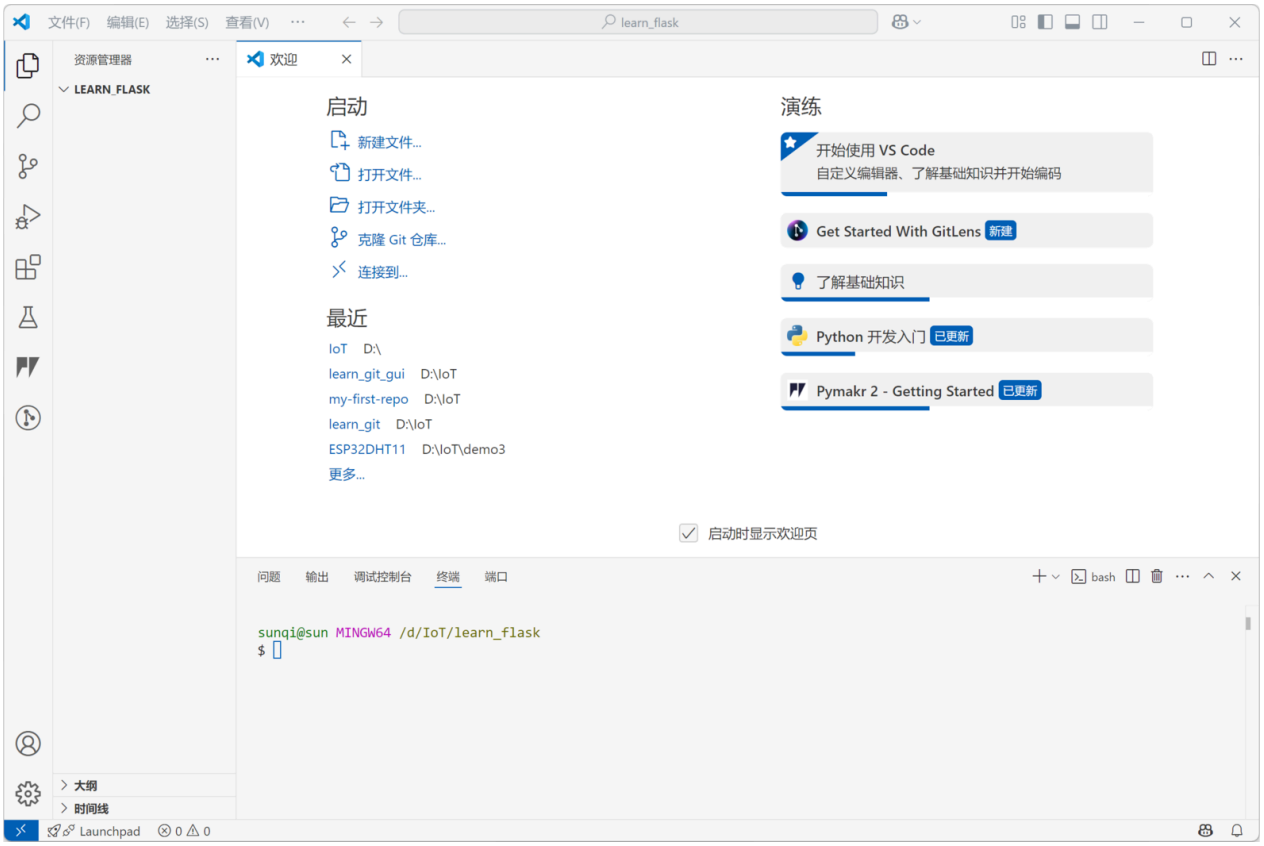
Flask 被称为"微框架"，因为它使用简单的核心，用扩展增加其他功能

Flask 特点

- **轻量级和简洁**：Flask 是一个微框架，提供了最基本的功能，不强制使用任何特定的工具或库。它的核心是简单而灵活的，允许开发者根据需要添加功能。
- **灵活性**：Flask 提供了基本的框架结构，但没有强制性的项目布局或组件，开发者可以根据自己的需求自定义。
- **可扩展性**：Flask 的设计允许你通过插件和扩展来添加功能。许多常见的功能，如表单处理、数据库交互和用户认证，都可以通过社区提供的扩展来实现。
- **内置开发服务器**：Flask 内置了一个开发服务器，方便在本地进行调试和测试。
- **RESTful 支持**：Flask 支持 RESTful API 的开发，适合构建现代的 Web 服务和应用程序。

快速入门

新建一个空文件夹 learn_flask，然后使用 vscode 打开这文件夹，作为项目目录



安装 flask

检查 flask 是否安装，在 vscode 终端运行

```Python
pip show flask
```

如果没有安装，则在 vscode 运行

```Bash
pip install flask
```

编辑代码

新建文件 app.py

```Python
from flask import Flask
app = Flask(__name__)

@app.route("/")
def hello():
    return "Hello World!"
```

```
if __name__ == "__main__":
```

运行代码

在终端运行

```
python app.py
```

浏览器预览

在浏览器输入：http://localhost:5000

代码解析：

- from flask import Flask： 这行代码从 `flask` 模块中导入了 `Flask` 类。`Flask` 类是 Flask 框架的核心，用于创建 Flask 应用程序实例。
- app = Flask(**name**)： 这行代码创建了一个 Flask 应用实例。`__name__`是一个特殊的 Python 变量，它在模块被直接运行时是`'__main__'`，在被其他模块导入时是模块的名字。传递`__name__`给`Flask` 构造函数允许 Flask 应用找到和加载配置文件。
- @app.route('/')： 这是一个装饰器，用于告诉 Flask 哪个 URL 应该触发下面的函数。在这个例子中，它指定了根 URL（即网站的主页）。
- def hello_world()::： 这是定义了一个名为 `hello_world` 的函数，它将被调用当用户访问根 URL 时。
- return 'Hello, World!'： 这行代码是 `hello_world` 函数的返回值。当用户访问根 URL 时，这个字符串将被发送回用户的浏览器。
- if **name** == '**main**'::这行代码是一个条件判断，用于检查这个模块是否被直接运行，而不是被其他模块导入。如果是直接运行，下面的代码块将被执行。
- app.run(debug=True)：这行代码调用 Flask 应用实例的 `run` 方法，启动 Flask 内置的开发服务器。`debug=True` 参数会启动调试模式，这意味着应用会在代码改变时自动重新加载，并且在发生错误时提供一个调试器。

物联网项目

文件目录结构

- app.py
- templates
  - index.html

- requirements.txt
- README.md

编写 app.py

Python

```python
from flask import Flask, render_template, request, redirect
import os

app = Flask(__name__)

# 全局变量存储目标温度
target_temperature = None

def get_latest_data():
    # 模拟获取温度和湿度数据
    temperature = 25.0  # 模拟温度
    humidity = 60.0     # 模拟湿度
    return temperature, humidity

@app.route('/')
def index():
    global target_temperature
    temperature, humidity = get_latest_data()
    return render_template('index.html', temperature=temperature, humidity=humidity, t
arget_temperature=target_temperature)

@app.route('/set-temperature', methods=['POST'])
def set_temperature():
    global target_temperature
    target_temp = request.form.get('target_temperature')
    if target_temp:
        try:
            target_temperature = float(target_temp)  # 更新目标温度
        except ValueError:
            print("无效温度输入")
    return redirect('/')

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5000))
    app.run(debug=True, host='0.0.0.0', port=port)
```

代码解读 app.py

Python

```
from flask import Flask, render_template, request, redirect
import os
```

- 从 Flask 库导入核心组件：
  - Flask：用于创建 Flask 应用实例。
  - render_template：用于渲染 HTML 模板文件（如 index.html）
  - request：用于获取前端提交的数据。
  - redirect：用于重定向用户请求到指定页面。
- 导入 os 模块，用于读取系统环境变量（如端口号）。

Python

```
app = Flask(__name__)
```

- 创建一个 Flask 应用实例，__name__是当前模块的名称。
- 这是启动 Flask 应用的标准写法。

Python

```
def get_latest_data():
    # 模拟获取温度和湿度数据
    temperature = 25.0  # 模拟温度
    humidity = 60.0     # 模拟湿度
    return temperature, humidity
```

- 定义一个函数 get_latest_data()，模拟从传感器获取当前温湿度。
- 这里使用了硬编码的假数据（温度 25.0℃，湿度 60%）。

Python

```
@app.route('/')
def index():
```

- 定义根路径/的页面响应，每次访问主页就会调用这个函数。

Python

```
    global target_temperature
    temperature, humidity = get_latest_data()
    return render_template('index.html', temperature=temperature, humidity=humidity, t
arget_temperature=target_temperature)
```

- 获取当前温湿度；
- 渲染 templates/index.html 页面，并传入温度、湿度、目标温度变量用于展示。

Python

```
@app.route('/set-temperature', methods=['POST'])
def set_temperature():
```

- 定义 POST 路由 /set-temperature，处理目标温度的设置。

```python
def set_temperature():
    global target_temperature
    target_temp = request.form.get('target_temperature')
    if target_temp:
        try:
            target_temperature = float(target_temp)  # 更新目标温度
        except ValueError:
            print("无效温度输入")
    return redirect('/')
```

- 获取表单中的 target_temperature 值。
- 将输入的值转换为浮点数并赋值给全局变量，如果输入不是数字，则打印错误信息。
- 设置完目标温度后，重定向回主页 /，刷新页面。

```python
if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5000))
    app.run(debug=True, host='0.0.0.0', port=port)
```

如果直接运行这个文件（而不是作为模块导入）：

- 从环境变量中读取端口（如果没有设置就默认 5000）；
- 启动 Flask 应用；
- host='0.0.0.0'表示允许局域网设备访问；
- debug=True 表示启用调试模式（便于开发）。

编写 templates/index.html

```html
<!doctype html>
<html lang="en">
<head>
  <title>IoT Dashboard</title>
</head>
<body>
  <h1>IoT Dashboard</h1>
  <div>
    <h4>当前温度：{{ temperature }} ℃</h4>
    <h4>当前湿度：{{ humidity }} %</h4>
    <h4>目标温度：{{ target_temperature }} ℃</h4>
  </div>
  <div>
    <form action="/set-temperature" method="POST">
    <label for="target_temperature">设置目标温度:</label><br>
    <input type="number" name="target_temperature" step="0.1" id="target_temperature"
placeholder="输入目标温度">
```

```
    <button type="submit">提交</button>
    </form>
  </div>
</body>
```

代码解读

```HTML
<!doctype html>
<html lang="en">
```

- 声明文档类型为 HTML5；
- lang="en"表示页面语言为英文（你也可以改成 zh 表示中文）。

```HTML
<head>
  <title>IoT Dashboard</title>
</head>
```

- 网页标题为“IoT Dashboard”，显示在浏览器的标签上。

```HTML
<body>
  <h1>IoT Dashboard</h1>
```

- 页面主标题，显示为大字号。

```HTML
<div>
  <h4>当前温度：{{ temperature }} ℃</h4>
  <h4>当前湿度：{{ humidity }} %</h4>
  <h4>目标温度：{{ target_temperature }} ℃</h4>
</div>
```

- 显示由 Flask 后端传过来的三个变量：
  - temperature：当前温度（比如 25.0）；
  - humidity：当前湿度（比如 60.0）；
  - target_temperature：目标温度（用户设定的值）；
- {{ }} 是 Jinja2 模板语法，用于把后端变量插入到 HTML 中。

```HTML
<div>
  <form action="/set-temperature" method="POST">
    <label for="target_temperature">设置目标温度:</label><br>
    <input type="number" name="target_temperature" step="0.1" id="target_temperature"
placeholder="输入目标温度">
```

```
    <button type="submit">提交</button>
  </form>
</div>
```

这是一个表单，让用户可以提交一个目标温度：

action="/set-temperature"：设置表单提交的目标路径，对应 Flask 后端的/set-temperature 路由；

method="POST"：使用 POST 方法提交数据；

<input type="number" ...>：输入框，只允许输入数字（可以带小数）；

step="0.1"允许输入小数；

placeholder="输入目标温度"是输入框的提示文字；

<button type="submit">提交</button>：点击后表单提交。

页面数据流逻辑（前后端配合）

1.用户访问/，Flask 渲染 index.html 页面；

2.后端将 temperature,humidity,target_temperature 变量传给模板；

3.页面展示这些数值；

4.用户在表单中输入新的目标温度，点击“提交”；

5.表单以 POST 方式提交到/set-temperature 路由；

6.Flask 后端接收表单数据，更新全局变量 target_temperature；

7.重定向回/，页面刷新，显示新设置的目标温度。

运行程序

```bash
python app.py
```

在浏览器访问：http://localhost:5000

文档整理

编写 requirements.txt

```bash
Flask
```

编写 README.md

```Bash
# IoT控制台
## 功能
- 监控环境温度
- 监控环境湿度
- 设置目标温度

## 运行
```bash
python app.py
```
```

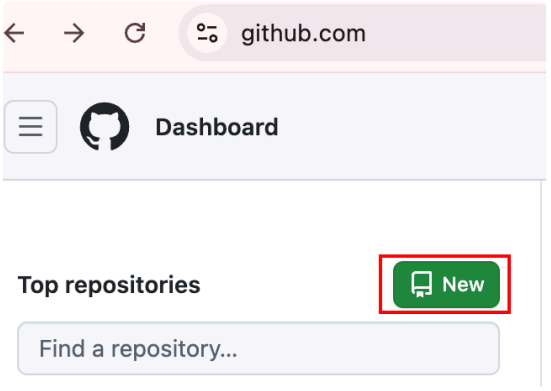
代码管理

在初始化仓库，添加文件，提交版本

```Bash
git init
git add app.py
git add templates/index.html
git add requirements.txt
git add README.md
git commit -m 'v0.1'
```

创建 Github 仓库并同步代码

1. 登录 github，创建新项目 IoT_Dashboard



Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? Import a repository.

Required fields are marked with an asterisk (*).

Owner * **Repository name ***

SuibeAI / IoT_Dashboard

IoT_Dashboard is available.

Great repository names are short and memorable. Need inspiration? How about **bug-free-barnacle** ?

Description (optional)

Public
Anyone on the internet can see this repository. You choose who can commit.

Private
You choose who can see and commit to this repository.

Initialize this repository with:

Add a README file
This is where you can write a long description for your project. Learn more about READMEs.

Add .gitignore

.gitignore template: None

Choose which files not to track from a list of templates. Learn more about ignoring files.

Choose a license

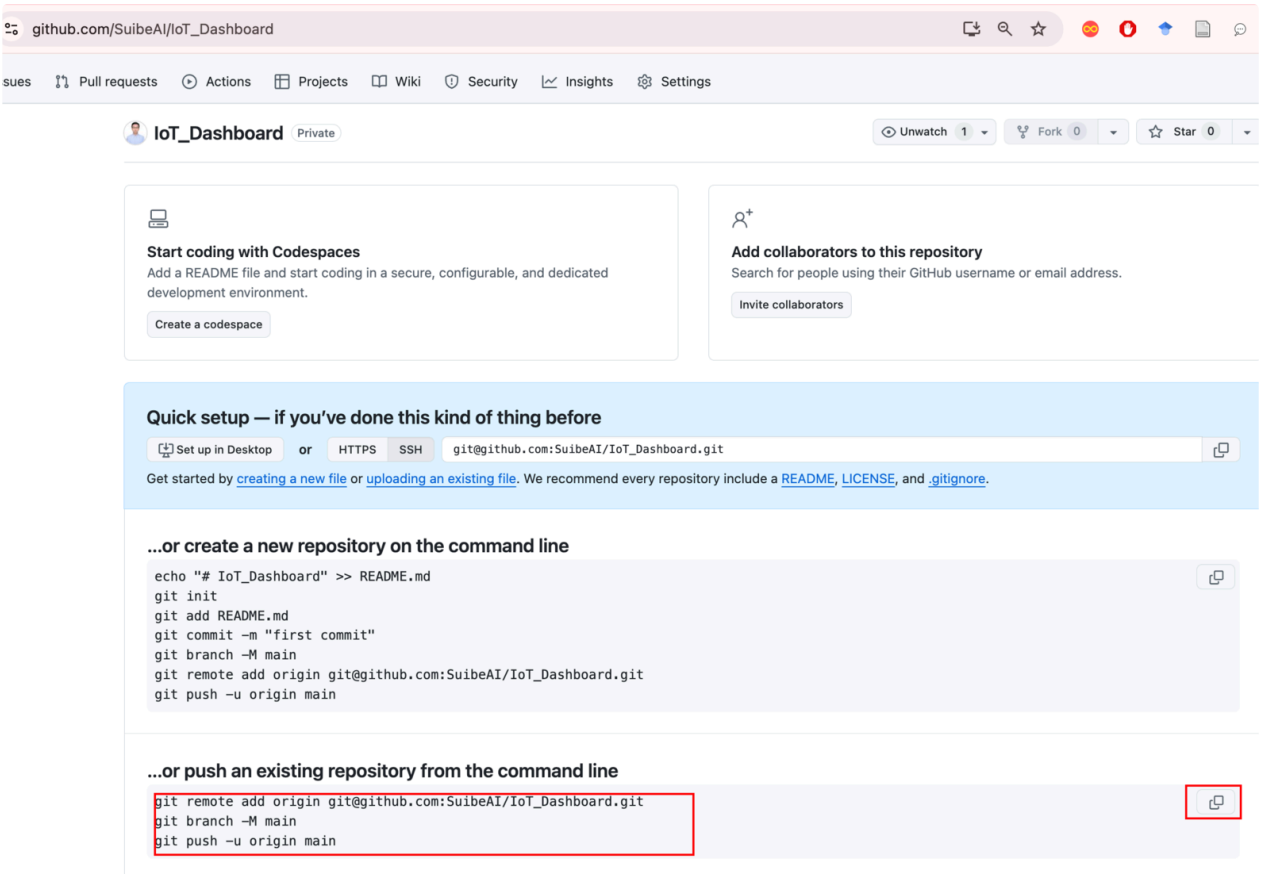
License: None

A license tells others what they can and can't do with your code. Learn more about licenses.

You are creating a private repository in your personal account.

Create repository

2. 同步代码

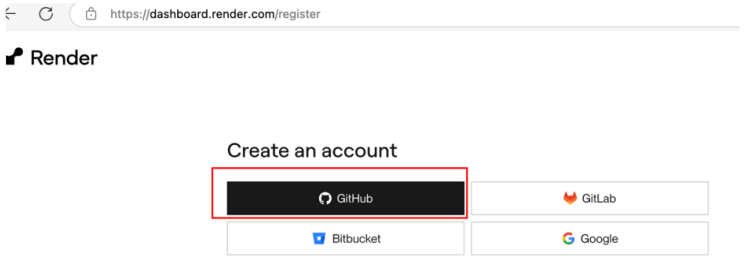


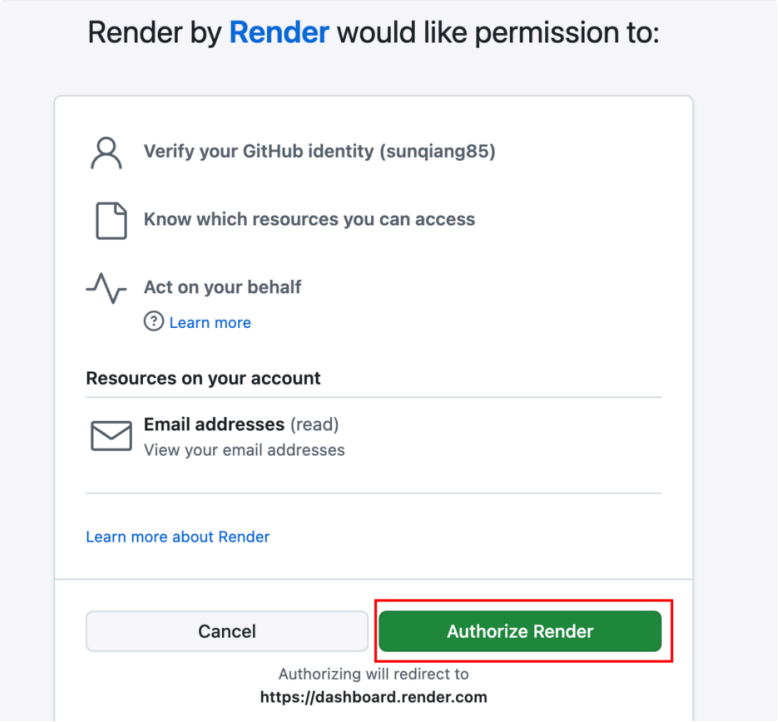
请根据红框修改为自己的仓库，即将 SuibeAI 替换成自己的用户名

```bash
git remote add origin git@github.com:SuibeAI/IoT_Dashboard.git
git branch -M main
git push -u origin main
```

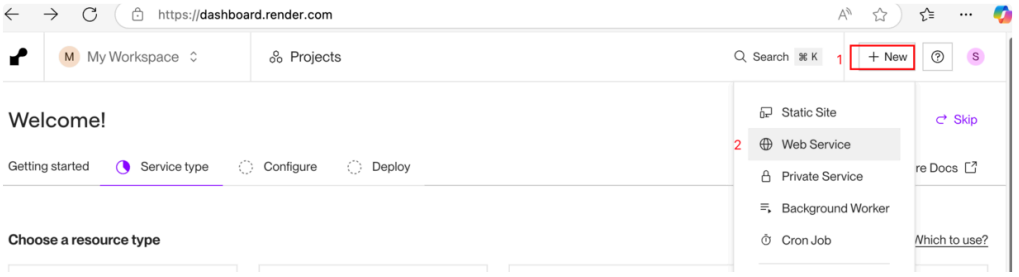
部署网站

1. 使用 github 登录 Render.com。

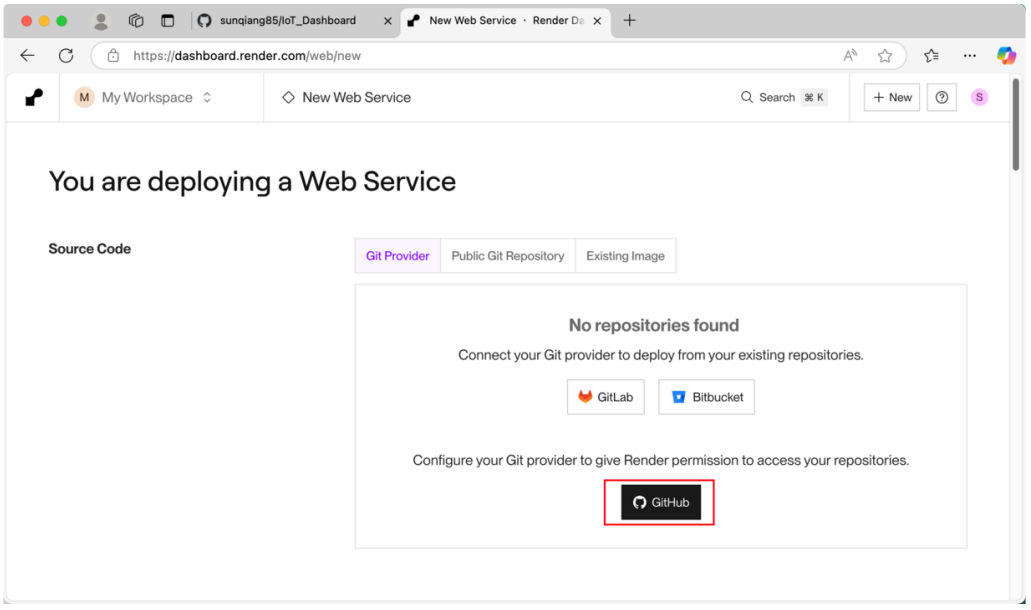


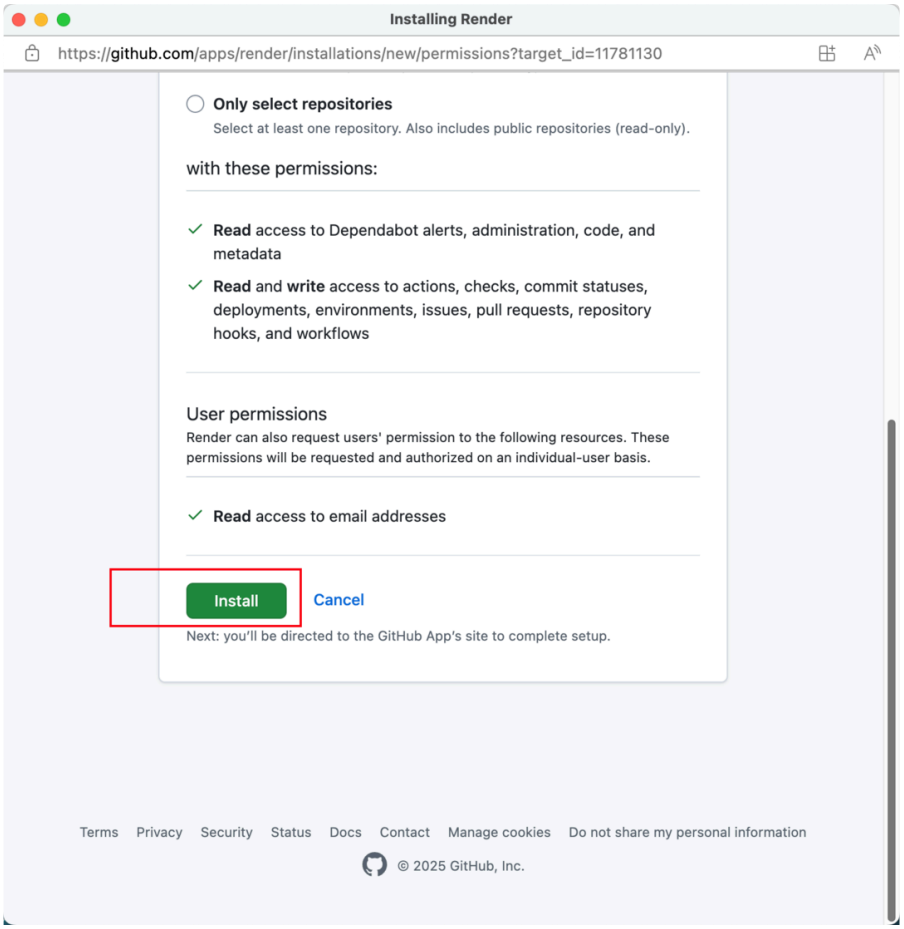


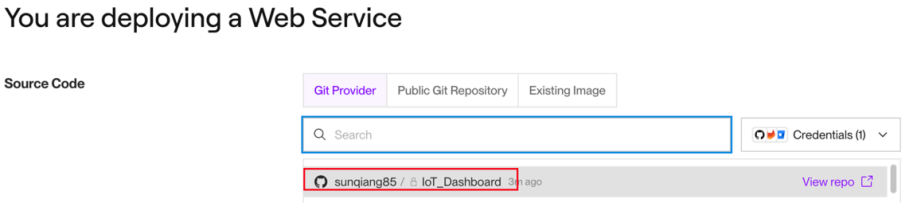
2. 点击 New + 按钮，选择 Web Service。



3. 连接你的 GitHub/GitLab 仓库，选择你的项目

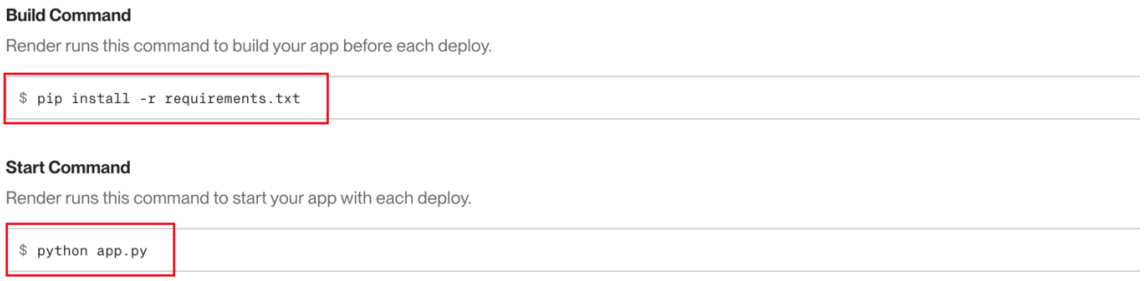


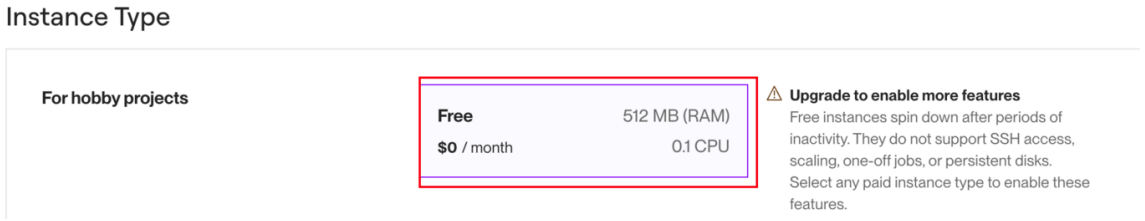




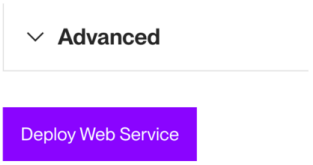
4. 配置服务：

- Build Command: `pip install -r requirements.txt`
- Start Command: `python app.py`





5. 部署服务



6. 等待部署完毕（约 1 分钟），负责网址，通过浏览器访问

